The Technical Documentation

IEEE1609.2.1 Standards-Compliant • Privacy-Preserving • Open

Techinical Documentation

LGEUS Advanced Security Team

December 30, 2025

1 Introduction

The **openSCMS** project is an open-source implementation of a Security Credential Management System (SCMS) compliant with the IEEE 1609.2 and IEEE 1609.2.1 standards for V2X communication.

Unlike traditional PKI systems, an SCMS must simultaneously satisfy strict cryptographic correctness, large-scale performance, and strong privacy requirements. In particular, the SCMS must enable authenticated and authorized communication between V2X end entities while preventing long-term tracking of vehicles and infrastructure nodes.

The openSCMS addresses this challenge by providing a “minimum complete” implementation of the core SCMS workflows defined by IEEE 1609.2.1. The system supports enrollment certificate provisioning and authorization certificate provisioning, including butterfly key expansion to compose pseudonym certificate generation.

From an engineering perspective, openSCMS is designed as a distributed, microservice-based system implemented primarily in Rust, with a dedicated cryptographic and protocol engine implemented in C. This separation allows the system to combine strong safety and concurrency guarantees at the service level with precise control over cryptographic operations and ASN.1 codecs operation at the protocol level.

This technical documentation complements the openSCMS White Paper by focusing on implementation details, repository structure, deployment procedures, APIs, and contribution guidelines, with the goal of enabling engineers to understand, execute, extend, and contribute to the openSCMS codebase.

2 Architecture Decisions

openSCMS follows a layered and service-oriented architecture aligned with the logical decomposition proposed by IEEE 1609.2.1. Each SCMS component is implemented as an independent Rust crate following microservices design, allowing clear separation of responsibilities, independent scaling, and reduced trust concentration. Based on that, V2X end entities (OBUs, RSUs, and test clients) can interact with the SCMS via IEEE 1609.2.1-defined REST interfaces.

2.1 High-Level Layers

The system can be viewed as four primary layers:

- **Service Layer**: Rust-based microservices implementing SCMS components such as RA, ECA, ACA, and LA.
- **Messaging and Persistence Layer**: Asynchronous task coordination via RabbitMQ (implemented in components that receive a higher load of frequent requests, such as RA) and persistent storage using relational databases (MySQL).
- **Cryptographic and Protocol Layer**: The `oscms-codecs-bridge` C library, responsible for cryptography, ASN.1 encoding/decoding, and SPDU parsing and construction.

2.2 Technology Choices

Rust Backend Rust is used for all SCMS services due to its strong guarantees around memory safety, concurrency, and performance properties that are essential for security-critical, highly concurrent systems handling sensitive cryptographic material.

An additional and decisive factor in this choice is Rust's clean and explicit integration model with C through Foreign Function Interfaces (FFI). This allows openSCMS to safely and efficiently interface with the `oscms-codecs-bridge`, a low-level C library that implements the core cryptographic and IEEE 1609.2.1 protocol logic. Rust bindings provide a clear boundary between high-level service orchestration and low-level cryptographic operations, enabling strong type safety, well-defined ownership semantics, and controlled memory management across the language boundary.

C Cryptographic Bridge The `oscms-codecs-bridge` is implemented in C to provide fine-grained control over memory layout, cryptographic operations and libraries, and ASN.1 processing. The bridge library is codec-agnostic and abstracts the underlying ASN.1 transpiler behind a stable API, avoiding dependency on proprietary tooling.

Microservices and Messaging The IEEE 1609.2.1 standard defines client vs SCMS interactions primarily through REST-based endpoints, making a microservice-oriented architecture a natural and practical design choice for openSCMS. By mapping each logical SCMS role to an independent service, the system closely mirrors the structure and interaction model prescribed by the standard.

Task parallelism is achieved through asynchronous messaging, enabling services to decouple request ingestion from computationally expensive cryptographic and certificate generation workflows. This design allows openSCMS to process large volumes of certificate requests without blocking synchronous API endpoints.

This approach is particularly important for high-demand components such as the Registration Authority (RA), where V2X clients frequently request batches of authorization or pseudonym certificates to support credential rotation. By offloading long-running tasks to message queues, the RA can scale horizontally, handle burst traffic efficiently, and maintain predictable response behavior even under heavy load.

This architecture ensures that no single component can compromise system-wide privacy guarantees while remaining practical for real-world deployments.

3 Repository Structure

The openSCMS repository is organized as a Rust workspace composed of multiple independent service crates, each implementing a specific SCMS role defined by IEEE 1609.2.1. The repository structure reflects a deliberate design choice to align code organization with system responsibilities, deployment boundaries, and trust separation.

Rather than a monolithic codebase, openSCMS adopts a *service-per-crate* model, where each SCMS component is compiled, deployed, and scaled independently.

3.1 Rust Workspace Organization

At the top level, the repository is a Cargo workspace that groups all SCMS services and shared libraries:

```
openscms/
├── aca/        # Authorization Certificate Authority service
├── eca/        # Enrollment Certificate Authority service
├── ra/         # Registration Authority service
├── la/         # Linkage Authority services
├── cam/        # Certificate Access Manager
├── dc/         # Device Configuration service
├── scmscommon/ # Shared Rust utilities and abstractions
├── certs/      # Certificates and keys required at runtime
├── deployment/ # Helm charts and Kubernetes manifests
├── docker/     # CI and build container definitions
├── scripts/    # Helper and bootstrap scripts
└── oscms_bridge/
    └── oscms-codecs-bridge/ # C cryptographic and protocol engine
```

Each directory under the workspace root corresponds to an independent Rust crate (expect certs, deployment, docker and scripts). Most of these crates (e.g., `ra/`, `eca/`, `aca/`, `la/`, `cam/`, `dc/`) are *service crates* that produce a single SCMS service binary, implementing a specific SCMS role and intended to be deployed as an independent microservice.

In addition to service crates, the workspace also includes shared *library crates*. Both `scmscommon` and `oscms_bridge` are Rust library crates (`crate-type = lib`) reused across multiple services. While `scmscommon` provides common Rust-level utilities and infrastructure abstractions, `oscms_bridge` acts as the Rust integration layer for the C-based `oscms-codecs-bridge`, encapsulating all Rust–C interoperability.

In addition to Rust crates, the repository includes a small set of directories dedicated to environment management and operational concerns rather than application logic. The `certs/` directory contains the certificates and keys required at runtime by SCMS services; `deployment/` provides Helm charts and Kubernetes manifests used for deployment; `scripts/` contains helper scripts for bootstrapping and automation tasks; and `docker/` includes container definitions used for CI and build pipelines.

3.2 Internal Structure of a Service Crate

Although each service implements a different SCMS role, their internal structure follows a common pattern, which simplifies navigation and onboarding:

- `main.rs`: Service entry point, responsible for startup, configuration loading, and HTTP server initialization.
- `endpoint.rs`: Definition of REST or binary endpoints exposed by the service.
- `core_logic/`: Implementation of the service's domain-specific logic, including certificate workflows and protocol handling.
- `entities/`: Data models representing persisted or derived SCMS entities.
- `persistence/`: Database access layer and query abstractions.

- `migrator/`: Schema migrations required by the service.

This separation enforces a clear distinction between transport and API concerns, protocol and cryptographic logic, and persistence and state management.

3.3 Role of `scmscommon`

The `scmscommon` crate is a Rust library crate that contains functionality shared across multiple SCMS services. This includes common configuration handling, shared data types, error abstractions, and utility routines that are orthogonal to any specific SCMS role.

By centralizing this functionality in a dedicated crate, openSCMS avoids code duplication while maintaining strict service boundaries. Importantly, `scmscommon` does not implement SCMS business logic or protocol workflows, ensuring that each service crate remains responsible only for its own role as defined by the standard.

3.4 The `oscms_bridge` Rust Crate

The `oscms_bridge` crate is a Rust library crate that provides a safe and idiomatic Rust interface to the C-based `oscms-codecs-bridge`. It serves as the explicit language and responsibility boundary between Rust services and the low-level cryptographic and protocol engine.

This crate contains a `build.rs` script responsible for:

- compiling the C sources of the `oscms-codecs-bridge`,
- linking the resulting static or shared library into the Rust build, and
- automatically generating Rust bindings for the C API using the `bindgen` tool.

By generating bindings at build time, `oscms_bridge` ensures that the Rust view of the C API remains consistent with the underlying implementation, while preserving Rust's safety guarantees through carefully designed wrapper types and ownership boundaries.

The `oscms_bridge` crate also contains the `oscms-codecs-bridge` subrepository, which is itself a fully independent C project. This C component is structurally and conceptually isolated from the Rust workspace and implements the core cryptographic and IEEE 1609.2.1 protocol logic of openSCMS.

The `oscms-codecs-bridge` is built using CMake and does not depend on Rust code or Rust-specific tooling. It exposes a well-defined C API that is consumed by multiple SCMS services via the `oscms_bridge` Rust lib crate, but remains entirely agnostic to service-level concerns and deployment details.

This strict separation allows the C bridge to be developed, audited, and reused independently of the Rust backend. The design, build system, and coding guidelines of the `oscms-codecs-bridge` are described in detail in a dedicated section later in this document.

3.5 Dependency Model and Service Boundaries

From a Rust perspective, service dependencies are intentionally minimal and explicit. Basically, each service depends on the `oscms_bridge` Rust crate, which in turn manages compilation, linking, and binding generation for the underlying C-based `oscms-`

`codecs-bridge` for cryptographic processing and IEEE 1609.2.1 SPDU handling. Also, no Rust service directly depends on another service's crate.

Inter-service coordination occurs exclusively through well-defined network APIs and asynchronous messaging, not through shared code or in-process calls. This enforces deployment-time isolation and mirrors real-world SCMS trust boundaries.

4 Deployment

This section describes how openSCMS can be deployed and executed in different environments, ranging from manual execution of individual components to a fully unified deployment of the complete SCMS ecosystem.

Each SCMS component in openSCMS (e.g., RA, ECA, ACA, LA) is built as an independent service and can, in principle, be deployed manually as a standalone process. In this model, the operator is responsible for ensuring that all external services required by the component are already deployed, properly configured, and reachable. These external dependencies include, but are not limited to:

- a relational database (MySQL) for persistent state, and
- a message queue service (RabbitMQ), required primarily by the Registration Authority (RA).

The deployment, installation, and configuration of these external services are considered out of scope for this document.

To simplify deployment and reduce operational complexity, openSCMS also provides a unified deployment approach based on Docker, Kubernetes, and Skaffold. This approach deploys the entire openSCMS ecosystem in a coordinated manner, including all SCMS service components, dedicated database instances for each component, and a RabbitMQ instance used by the RA.

This unified deployment mechanism is the **recommended** way to run openSCMS, particularly for development, testing, and evaluation purposes, as it ensures a consistent environment and minimizes configuration errors.

Before describing concrete deployment examples, the following subsections outline the common prerequisites required by both manual and unified deployments.

4.1 Build Dependencies

The openSCMS relies on a combination of Rust tooling, native system libraries, and container-orchestration utilities. The required dependencies can be divided into two categories: dependencies required to build and run individual SCMS components, and dependencies required for unified ecosystem deployment.

4.1.1 Core Build and Runtime Dependencies

The following tools and libraries are required to build and execute individual openSCMS components (e.g., RA, ECA, ACA) manually:

- Rust toolchain (`rustc`, `cargo`, `rustfmt`, `clippy`)
- CMake

- `libclang-dev`
- `llvm-dev`
- `libzip-dev`
- `libssl-dev`

These dependencies are required to compile both the Rust service crates and the C-based `oscms-codecs-bridge`, as well as to generate Rust bindings via `bindgen`.

4.1.2 Unified Deployment Dependencies

For deploying the complete openSCMS ecosystem using the recommended unified approach, the following additional tools are required:

- `helm`
- `kubectl`
- `kubeseal`
- `minikube`
- `skaffold`
- `sea-orm-cli` (for database migrations and schema management)

openSCMS uses `binenv` to manage consistent versions of Kubernetes-related tooling. To install `binenv`, follow the official instructions, which can be summarized as:

```
wget -q -O binenv https://github.com/devops-works/binenv/releases/download/v0.1
chmod +x ./binenv
./binenv update
./binenv install binenv
rm ./binenv
echo 'export PATH=$HOME/.binenv:$PATH' >> $HOME/.bashrc
echo 'source <(binenv completion bash)'
exec $SHELL
```

Once `binenv` is installed, install all required tools. The exact versions are pinned in the `.binenv.lock` file at the repository root:

```
binenv install -l
```

From this point on, entering the repository directory will automatically expose the required tools in the shell `PATH`.

A complete reference environment, including all required dependencies, is also documented in `docker/lgscms-ci.dockerfile`.

4.2 Deployment Parameters and Environment Variables

openSCMS is configured entirely through environment variables. These parameters control deployment mode, logging behavior, service ports, cryptographic policies, protocol behavior, and the location of certificates and protocol artifacts.

The complete list of parameters, along with a detailed explanation of their semantics and default values, is provided in a dedicated appendix (Appendix Section A).

4.3 Component Certificates and Keys: certs folder

Before deploying openSCMS, a complete and consistent set of certificates and cryptographic keys must be generated for all internal SCMS components and external trust anchors. While certificate generation is formally outside the scope of the openSCMS runtime, it is a mandatory prerequisite for any functional deployment.

All SCMS services expect certificates and keys to be available locally at startup, following a predefined directory structure under the `certs/` folder. Missing or inconsistent cryptographic material will prevent services from starting correctly.

To simplify certificate and key generation for development and testing deployments, openSCMS provides the `scms_manager` utility. This tool is located within the C-based `oscms-codecs-bridge` project:

```
openscms/oscms_bridge/oscms-codecs-bridge/scms_manager
```

The `scms_manager` generates all cryptographic payloads required by openSCMS services, including component certificates, public and private keys, and protocol-specific artifacts. It also produces external trust material used by V2X clients, such as elector certificates and Root CA keys.

4.3.1 Generating Certificates

From the repository root, the following script can be used to invoke the `scms_manager` and populate the `certs/` directory automatically:

```
sh scripts/run_tests_scms_manager.sh
```

After execution, the `certs/` directory is populated with the expected structure and files:

```
certs/
├── aca
│   ├── acaCertificate.coer
│   ├── acaPrivateKey.dat
│   └── acaPublicKey.dat
├── ctl
│   ├── ccf.coer
│   └── ctl.coer
├── eca
│   ├── ecaCertificate.coer
│   ├── ecaPrivateKey.dat
```

```
│   └── ecaPublicKey.dat
├── ica
│   └── icaCertificate.coer
├── ra
│   ├── raCertificate.coer
│   ├── raEncPrivateKey.dat
│   ├── raEncPublicKey.dat
│   ├── raPrivateKey.dat
│   └── raPublicKey.dat
└── rootca
    └── rootcaCertificate.coer
```

4.3.2 External Service Files

In addition to internal SCMS component certificates, the `scms_manager` generates external trust artifacts that are not consumed directly by openSCMS services but are essential for V2X clients.

These files include elector certificates and Root CA material, which together form part of the V2X client root of trust:

```
external_service_files/
├── electors_files
│   ├── 1_electorCertificate.coer
│   ├── 1_electorPrivateKey.dat
│   ├── 1_electorPublicKey.dat
│   ├── 2_electorCertificate.coer
│   ├── 2_electorPrivateKey.dat
│   ├── 2_electorPublicKey.dat
│   ├── 3_electorCertificate.coer
│   ├── 3_electorPrivateKey.dat
│   └── 3_electorPublicKey.dat
├── ica_files
│   ├── icaCertificate.coer
│   ├── icaPrivateKey.dat
│   └── icaPublicKey.dat
└── root_ca_files
    ├── rootcaCertificate.coer
    ├── rootcaPrivateKey.dat
    └── rootcaPublicKey.dat
```

4.4 Manual Deployment Example: Registration Authority (RA)

This subsection illustrates a manual deployment of a single openSCMS component using the Registration Authority (RA) as a concrete example. The same general procedure applies to other SCMS services, with variations only in required external dependencies and configuration parameters.

Manual deployment is primarily intended for development, debugging, and advanced experimentation scenarios. For most use cases, the unified deployment approach described later in this section is strongly recommended.

4.4.1 Prerequisites and Environment Setup

Before starting the RA service manually, the following conditions must be met:

- All build dependencies described earlier in this section are installed.
- The `certs/` directory is populated with valid certificates and keys, following the structure described in Section 4.3.
- A MySQL instance is running and reachable by the RA service.
- A RabbitMQ instance is running and reachable by the RA service.

The deployment, installation, and configuration of MySQL and RabbitMQ are outside the scope of this document.

For this example, the following connection strings are assumed:

```
MYSQL_CONNECTION_STRING=mysql://username:password@mysql:3306
RABBITMQ_CONNECTION_STRING=amqp://rmqadmin:password@scmsrabbitmq:5672
```

In addition to the global deployment parameters described earlier, the RA requires database and messaging configuration to be provided via environment variables. At a minimum, the following variables must be defined:

```
MYSQL_USERNAME=username
MYSQL_PASSWORD=password
MYSQL_HOST=mysql
MYSQL_PORT=3306
MYSQL_CONNECTION_STRING=mysql://username:password@mysql:3306

RABBITMQ_CONNECTION_STRING=amqp://rmqadmin:password@scmsrabbitmq:5672
```

4.4.2 Building and Running the RA Service

Once the environment is correctly configured, the RA service can be built and started.

From the repository root, build the complete Rust workspace:

```
cargo build
```

This step compiles all Rust service crates as well as the C-based `oscms-codecs-bridge`, and generates the required Rust bindings via `bindgen`. Build failures at this stage typically indicate missing system dependencies or misconfigured toolchains.

To start the Registration Authority service, change into the RA crate directory and run:

```
cd openscms/ra
cargo run
```

Upon startup, the RA service will load its certificates and keys, establish connections to the configured MySQL database and RabbitMQ broker, and expose its REST endpoints on the configured port.

Startup failures usually result from missing certificates, incorrect environment variables, or unavailable external services.

4.5 Unified Deployment: The Easy Way

While manual deployment is useful for development and debugging, the recommended way to run openSCMS is through its unified deployment mechanism based on Docker, Kubernetes, and Skaffold. This approach deploys the complete openSCMS ecosystem in a single, coordinated workflow, including all SCMS services, their databases, and supporting infrastructure such as RabbitMQ.

For local development and evaluation, openSCMS provides a reference deployment based on Minikube.

4.5.1 Environment Setup Using Minikube

Installing Required Tooling All required tooling should be installed using `binenv`, as described in Subsection 4.1.2. This ensures that consistent and compatible versions of all Kubernetes-related tools are used across different development environments.

Configuring Minikube openSCMS uses the Docker driver for Minikube. This can be configured permanently by editing (or creating) the Minikube configuration file with the following content:

```
{
  "vm-driver": "docker"
}
```

Installing kubectl Plugins To simplify cluster management, install the `krew` plugin manager for `kubectl`:

```
(
  set -x; cd "$(mktemp -d)" &&
  OS="$(uname | tr '[:upper:]' '[:lower:]')" &&
  ARCH="$(uname -m | sed -e 's/x86_64/amd64/' \
                         -e 's/\(arm\)\(64\)\?.*/\1\2/' \
                         -e 's/aarch64$/arm64/')" &&
  KREW="krew-${OS}_${ARCH}" &&
  curl -fsSLO "https://github.com/kubernetes-sigs/krew/releases/latest/download
  tar zxvf "${KREW}.tar.gz" &&
  ./"${KREW}" install krew
)
echo 'export PATH="$HOME/.krew/bin:$PATH"' >> $HOME/.bashrc
exec $SHELL
```

Verify the installation:

```
kubectl krew --help
```

Finally, install the RabbitMQ plugin to simplify inspection and configuration of the message broker:

```
kubectl krew install rabbitmq
```

4.5.2 Starting Minikube and Deploying openSCMS

Start Minikube with:

```
minikube start
```

The first startup may take several minutes as container images are downloaded.

Once Minikube is running, perform a test deployment of openSCMS from the repository root:

```
skaffold run && skaffold delete
```

This command deploys the full SCMS stack and then immediately tears it down, verifying that all components can be built and scheduled correctly.

After the deployment completes, verify that the RabbitMQ components were installed successfully:

```
kubectl get ns
```

A namespace named `rabbitmq-system` should be present.

4.5.3 Accessing Services via Minikube

Minikube exposes services using an IP address that can be obtained with:

```
minikube ip
```

For convenience, this IP address can be mapped to a hostname by adding an entry to `/etc/hosts`:

```
sudo echo "$(minikube ip) minikube" >> /etc/hosts
ping minikube
```

This allows services to be accessed using a stable hostname during development and testing.

5 Core SCMS Components

openSCMS implements the core functional entities defined by the IEEE 1609.2.1 Secure Credential Management System (SCMS) architecture. Each entity is realized as an independent Rust crate, deployed as a microservice, and responsible for a well-defined subset of the SCMS protocol flows.

The primary SCMS components implemented in openSCMS are:

- Registration Authority (RA)
- Enrollment Certificate Authority (ECA)
- Authorization Certificate Authority (ACA)
- Linkage Authority (LA)

Among these components, the RA and ECA expose external REST APIs and act as ingress points for V2X clients. The ACA and LA are internal infrastructure services that do not expose public APIs and are accessed exclusively by other SCMS components.

For each component, this section describes its architectural role, its responsibilities within the SCMS protocol, and the main operational flows in which it participates. The discussion starts with the Registration Authority (RA), which serves as the primary entry point into the SCMS ecosystem.

5.1 Registration Authority (RA)

The Registration Authority (RA) is the primary front-end component of openSCMS and the main interaction point for V2X clients. For architectural simplicity, the RA consolidates the roles of the Device Configuration (DC) and Device Configuration Manager (DCM) defined in IEEE 1609.2.1.

In practice, this makes the RA responsible for End Entity (EE) lifecycle management, distribution of trust and configuration artifacts, and coordination of certificate provisioning workflows with other SCMS components.

5.1.1 Internal Organization and Execution Model

The RA is internally organized into two deployment-time roles: the *RA Frontend* and the *RA Worker*. The RA Frontend exposes the external REST interface and handles all client-facing protocol logic, while RA Workers execute SCMS operations asynchronously.

The RA Frontend validates incoming SPDUs, verifies signatures, decrypts encrypted payloads when required, and constructs internal tasks that are enqueued for processing. It responds immediately to the client with an acknowledgment SPDU indicating when the client should retry the corresponding download request.

RA Workers monitor the task queue and execute tasks independently. They interact with internal SCMS services such as the ACA and ECA, perform cryptographic processing, and persist results to the database using the request hash as the primary correlation key.

5.1.2 Authorization Certificate Provisioning Flow

The Authorization Certificate provisioning flow is responsible for issuing batches of authorization (pseudonym) certificates that are used by V2X clients for authenticated application-layer communication.

This flow is intentionally split into two phases: a request phase and a download phase. The request phase is synchronous and lightweight, allowing the RA to validate and acknowledge the request quickly. The actual certificate generation is performed asynchronously to support high request volumes and batch-based issuance. The download phase allows the client to retrieve the generated certificates once processing has completed.

From an architectural perspective, the RA Frontend terminates the client connection, validates and decrypts the request, and enqueues a task for asynchronous execution. One of the RA Worker instances then processes the task by invoking the Authorization Certificate Authority (ACA), persists the generated certificates, and makes them available for download.

Authorization Certificate Request The client submits an Authorization Certificate Request SPDU to the RA Frontend. The request contains cryptographic material and authorization parameters required to generate one or more authorization certificates.

```
POST /v3/authorization-certificate

Description:
  Submits a request for one or more authorization (pseudonym)
  certificates.

Input:
  - IEEE 1609.2.1 Authorization Certificate Request SPDU
  - Contains enrollment reference, butterfly key parameters,
    certificate permissions, and validity constraints

Output:
  - Acknowledgment SPDU (ACK)
  - Includes timing information indicating when the client
    should issue the download request

Errors:
  - Invalid or malformed SPDU
  - Signature verification failure
  - Decryption failure
  - Policy or parameter violations
```

Authorization Certificate Download

```
GET /v3/authorization-certificate

Description:
  Retrieves the batch of authorization certificates generated
  for a previously submitted request.
```

```
Input:
  - Download Request SPDU
  - Includes request hash returned during the request phase

Output:
  - Response SPDU containing a batch of authorization
    (pseudonym) certificates

Errors:
  - Unknown or expired request hash
  - Certificates not yet available
```

5.1.3 Successor Enrollment Certificate Flow

The Successor Enrollment Certificate flow allows an already enrolled End Entity to obtain a new enrollment certificate while preserving sensitive identity information. This flow is used to renew enrollment credentials without re-running the initial enrollment process.

As with authorization provisioning, the flow is divided into a request phase and a download phase. The request phase validates and decrypts the incoming enrollment data and schedules certificate generation asynchronously. The Enrollment Certificate Authority (ECA) performs the actual issuance of the new enrollment certificate, which is then stored by the RA and made available for download.

The RA acts as the cryptographic ingress point for this flow, decrypting the request and enforcing lifecycle and policy constraints before forwarding it to the ECA.

Successor Enrollment Certificate Request

```
POST /v3/successor-enrollment-certificate

Description:
  Submits a request for a successor enrollment certificate.

Input:
  - Encrypted IEEE 1609.2.1 Successor Enrollment Request SPDU
  - Contains identity-related enrollment parameters

Output:
  - Acknowledgment SPDU (ACK)
  - Indicates when the client should issue the download request

Errors:
  - Decryption failure
  - Invalid enrollment state
  - Malformed request SPDU
```

Successor Enrollment Certificate Download

```
GET /v3/successor-enrollment-certificate
```

```
Description:
  Retrieves the successor enrollment certificate generated
  for a previous request.

Input:
  - Download Request SPDU
  - Includes request hash

Output:
  - Response SPDU containing the new enrollment certificate

Errors:
  - Unknown request hash
  - Certificate not yet available
```

5.1.4 EE Registration and Policy Distribution

Before an End Entity (EE) can participate in enrollment or certificate provisioning flows, it must first be registered with the SCMS. In openSCMS, this initial step is handled by the Registration Authority (RA), acting in its combined DC/DCM role.

The registration process establishes the EE identity within the system, associates a canonical identifier with a device identifier, and returns the initial policy that governs which SCMS operations the EE is allowed to perform. This flow is synchronous and is typically executed during initial device provisioning or manufacturing.

EE Registration Request

```
POST /dcms/device/na

Description:
  Registers a new End Entity (EE) in the SCMS.

Input:
  - JSON payload containing EE metadata and identifiers

Output:
  - Confirmation of successful registration

Errors:
  - Duplicate EE registration
  - Invalid or incomplete metadata
```

The EE registration request is a JSON-based request that provides the RA with the minimal information required to identify and register a new device.

Request Payload Example:

```
{
  "deviceType": "obu",
  "publicKey": "04e9fed3f1a32e52ec...d0",
  "version": "na"
```

```
}
```

Semantics: The request identifies the type of EE (e.g., OBU or RSU) and provides a public key that will be associated with the device during its lifecycle. The version field allows compatibility handling across SCMS protocol versions.

Invalid payloads, unsupported device types, or malformed keys result in an error response and no state change.

EE Registration Response Upon successful registration, the RA creates a new EE record and returns a registration response containing the identifiers assigned to the EE and the policy that governs its interaction with the SCMS.

Response Payload Example:

```
{
  "canonicalId": "1f81f9...49f225fb248c3",
  "deviceId": "1587425867723428852",
  "deviceType": "obu",
  "publicKey": "04e0991d6ab1e94....4d2e2616b1e8774b52cae53",
  "status": "Registered",
  "version": "na",
  "createdTime": "2025-12-26T13:59:31.627873478+00:00",
  "updatedTime": "2025-12-26T13:59:31.627882625+00:00",
  "devicePolicy": { ... }
}
```

Semantics: The response assigns a stable `canonicalId`, which is used across SCMS cryptographic workflows, and a `deviceId`, which uniquely identifies the EE within the SCMS database. The response also embeds the EE policy file that specifies which protocol endpoints and certificate flows the device is authorized to invoke.

At this stage, the EE is considered `Registered` but not yet `Enrolled`. Subsequent enrollment and certificate provisioning flows depend on the policy returned during registration.

Policy File Download

```
GET /dcms/policy/na

Description:
  Returns policy and configuration information applicable
  to the requesting EE.

Input:
  - EE identifier and context information

Output:
  - Policy definition governing allowed SCMS operations
```

The Registration Authority exposes a single endpoint for policy retrieval. The policy returned depends on whether the request identifies a specific End Entity (EE).

If the request includes either the `canonicalId` or `deviceId` query parameter, the RA returns a device-specific policy that defines which SCMS operations the EE

is authorized to perform. If no EE-identifying parameters are provided, the RA returns the global SCMS policy, which defines system-wide configuration, trust anchors, and cryptographic constraints.

This unified design allows clients to retrieve both system-level and device-level policies using a consistent interface.

Device-Specific Policy When the request includes `canonicalId` or `deviceId`, the RA returns the policy associated with that specific EE. This policy controls which protocol flows the EE may invoke, as well as operational constraints such as certificate limits and supported regions.

Example Request:

```
GET /dcms/device/na?canonicalId=1f81a656b75fa5df95eea79824a0cef7bd64bad3
```

Example Fields Returned:

```
{
  "ee-scmsV3-eeAuth": {
    "authorization-certificate-request": ["Enrollment"],
    "authorization-certificate-download": ["Enrollment"],
    "successor-enrollment-certificate-request": ["Enrollment"]
  },
  "ee-countCert": 1,
  "ee-heartbeatInterval": 0
  ...
}
```

A complete example of the device-specific policy is provided in Appendix B.

Global SCMS Policy When no EE-identifying query parameters are provided, the RA returns the global SCMS policy. This policy defines parameters and constraints that apply uniformly across all registered EEs, including cryptographic algorithms, trust anchors, endpoint locations, and protocol behavior.

Example Request:

```
GET /dcms/device/na
```

Example Fields Returned:

```
{
  "quorumCount": 3,
  "ctlSeriesId": "FFFFFFFFFFFFFF00",
  "cryptoAlgorithm": [ "nistP256", "nistP384", "sha256", "sha384" ],
  "certRootCa": { ... },
  "certEca": { ... },
  "certAca": { ... },
  "raUrl": "http://192.168.49.2:30080/v3/",
  "ecaUrl": "http://192.168.49.2:30085/v3/",
  "scmsV3-error": "Fine",
  "scmsV3-eeAuth": { ... }
}
```

A complete example of the global SCMS policy is provided in Appendix C.

Semantics: The policies files provides the authoritative configuration required by the EE to interact correctly with the SCMS, including cryptographic constraints, endpoint locations, timing parameters, and authorization rules. Clients are expected to cache and periodically refresh this file according to their operational needs.

5.1.5 Trust Artifact Distribution

The RA also exposes endpoints for distributing trust and configuration artifacts required by V2X clients.

Endpoints include:

- `GET /v3/ra-certificate`
- `GET /v3/ca-certificate`
- `GET /v3/ctl`
- `GET /v3/ccf-ctl`
- `GET /v3/crl`
- `GET /v3/composite-crl-ctl`

These endpoints return protocol-compliant artifacts such as CA certificates, Certificate Trust Lists (CTLs), and Certificate Revocation Lists (CRLs). Errors are reported using SCMS-defined error semantics.

5.1.6 Cryptographic Responsibilities and Trust Role

The RA is the only openSCMS component whose certificate includes a public encryption key intended for use by V2X clients. This design makes the RA the cryptographic ingress point for sensitive operations.

Consequently, the RA decrypts and validates the majority of client-originated messages and verifies digital signatures across most protocol flows. During startup, the RA loads its own certificates as well as the certificates of other SCMS components from the `certs/` directory, enabling it to distribute trust material as part of its DC/DCM responsibilities.

5.1.7 End Entity Lifecycle Management

The Registration Authority (RA) is the authoritative component responsible for maintaining the lifecycle state of each End Entity (EE) within the SCMS. The lifecycle state represents the EE's progression through registration, enrollment, and certificate provisioning stages, and is used as a hard precondition for allowing or denying protocol flows.

Lifecycle state is explicitly stored and managed by the RA and is queried by other SCMS components to enforce protocol correctness. State transitions are never implicit and only occur as a result of successful execution of specific SCMS flows.

Lifecycle States The following lifecycle states are supported:

- `Registered`: The EE has been registered in the SCMS but does not yet possess a valid enrollment certificate.
- `Enrolled`: The EE possesses a valid enrollment certificate and is eligible to request authorization certificates.
- `Provisioning`: The EE is actively provisioning authorization certificates.
- `SuccessorEnrolled`: The EE has successfully completed a successor enrollment flow and holds a renewed enrollment certificate.
- `Blocked`: The EE is explicitly blocked and is not allowed to participate in any SCMS flows.
- `Deleted`: The EE has been removed from active use and all SCMS interactions are denied.

Valid Lifecycle Transitions The RA enforces the following valid lifecycle transitions:

- `Registered` → `Enrolled` Triggered after a successful enrollment certificate issuance by the ECA.
- `Enrolled` → `Provisioning` Triggered when the RA accepts an authorization certificate request from the EE.
- `Registered` → `SuccessorEnrolled` Triggered after a successful successor enrollment flow.
- `Provisioning` → `SuccessorEnrolled` Triggered after a successful successor enrollment flow while provisioning.
- `*` → `Blocked` Triggered by administrative action.
- `*` → `Deleted` Triggered by administrative action.

All other transitions are considered invalid and are rejected by the RA.

Lifecycle Enforcement Across SCMS Components The lifecycle state maintained by the RA is consulted by other SCMS components as part of their flow validation logic. In particular:

Before issuing an enrollment certificate, the Enrollment Certificate Authority (ECA) queries the RA using:

```
GET /dcms/device/na
```

The ECA verifies that the EE is in the `Registered` state. Enrollment requests originating from EEs in any other state are rejected.

Similarly, when the RA receives an authorization certificate request, it checks that the EE is in the `Enrolled` state. Upon accepting the request, the RA updates the EE lifecycle state to `Provisioning`.

For successor enrollment flows, the RA verifies that the EE is either in the `Registered` or `Provisioning` state. Upon successful issuance of a successor enrollment certificate, the lifecycle state is updated to `SuccessorEnrolled`.

Lifecycle State Updates Lifecycle transitions are persisted via the following endpoint:

```
PATCH /dcms/device/na
```

The request must identify the EE using the `deviceId` or `canonicalId` query parameter and provide the new lifecycle state explicitly in the request body.

Example Payload:

```
{
  "status": "Enrolled"
}
```

Error Handling and Invalid States If a protocol request is received that is incompatible with the current lifecycle state of the EE, the request is rejected with a protocol-defined error response. Examples include:

- Enrollment certificate requests from EEs not in the `Registered` state.
- Authorization certificate requests from EEs not in the `Enrolled` state.
- Any SCMS operation originating from an EE in the `Blocked` or `Deleted` state.

By centralizing lifecycle state management and enforcement within the RA, openSCMS ensures consistent protocol behavior, clear state transitions, and robust error handling across all SCMS components, even in distributed and horizontally scaled deployments.

5.2 Enrollment Certificate Authority (ECA)

The Enrollment Certificate Authority (ECA) is responsible for issuing enrollment certificates, which establish the long-term cryptographic identity of an End Entity (EE) within the SCMS. Enrollment certificates bind sensitive device identity information to cryptographic material and serve as the foundation for all subsequent authorization and provisioning flows.

Within the openSCMS architecture, the ECA participates directly in the EE lifecycle by transitioning devices from the `Registered` state to the `Enrolled` state, and by issuing renewed enrollment certificates during successor enrollment. The ECA does not maintain EE state locally; instead, it relies on the Registration Authority (RA) as the authoritative source of lifecycle information. Before issuing any enrollment certificate, the ECA queries the RA to validate that the EE is in a state compatible with the requested operation.

The ECA exposes REST endpoints for enrollment-related operations, but their usage is tightly constrained by lifecycle and policy checks enforced in coordination with the RA. This design minimizes the ECA's exposure surface while preserving a clear separation of responsibilities between identity issuance and lifecycle control.

5.2.1 Initial Enrollment Certificate Flow

The initial enrollment flow is responsible for issuing the first enrollment certificate for an EE. This operation represents a critical trust boundary within the SCMS, as enrollment certificates embed long-term identity attributes that are later used to authorize all other certificate provisioning activities.

From a cryptographic perspective, the initial enrollment request establishes the first authenticated link between the End Entity (EE) and the SCMS. The enrollment request submitted by the client must be digitally signed using the EE's *canonical private key*. This key corresponds to the canonical public key previously provided during the EE registration process and stored by the Registration Authority (RA).

Upon receiving an enrollment request, the ECA does not directly trust the cryptographic identity asserted by the client. Instead, the ECA queries the RA to retrieve the canonical information associated with th requesting EE, including the canonical public key. Using this information, the ECA verifies the digital signature of the enrollment request, ensuring that the request originates from the registered EE and has not been tampered with.

If signature verification succeeds and all lifecycle and policy preconditions are satisfied, the ECA issues an explicit IEEE 1609.2.1 enrollment certificate. This certificate is signed by the ECA and binds the EE's canonical identity to a new cryptographic credential that will be used for all subsequent interactions with the SCMS.

The enrollment certificate represents the transition from an unauthenticated registration context to an authenticated operational context. Once issued, it becomes the primary credential used by the EE to authenticate future protocol messages. For example, when the RA receives an authorization certificate request, it first validates the enrollment certificate by verifying its signature using the ECA certificate. Only after the enrollment certificate is validated does the RA verify the digital signature of the incoming request using the public key contained in the enrollment certificate.

Enrollment Certificate Request

```
POST /v3/enrollment-certificate

Description:
Submits a request for an initial enrollment certificate.

Input:
An IEEE~1609.2.1 Enrollment Certificate Request SPDU containing canonical
identity information and cryptographic material, signed using the EE's
canonical private key.

Output:
A response SPDU containing the newly issued enrollment certificate.

Errors:
Requests originating from EEs in invalid lifecycle states, malformed
SPDUs, signature verification failures, or cryptographic validation
errors result in protocol-defined error responses.
```

5.2.2 Security and Trust Considerations

The ECA operates on highly sensitive identity data and is therefore designed to remain narrowly scoped. It does not perform EE registration, lifecycle persistence, or authorization policy enforcement directly. These responsibilities are delegated to the RA, which acts as the single source of truth for EE state and eligibility.

By enforcing strict preconditions through RA coordination and limiting direct client interaction, the ECA ensures that enrollment certificates are issued only under well-defined, auditable conditions, preserving the security and privacy guarantees of the SCMS.

5.3 Authorization Certificate Authority (ACA)

The Authorization Certificate Authority (ACA) is responsible for issuing authorization certificates, also referred to as pseudonym certificates, which are used by End Entities (EEs) to authenticate and sign V2X application-layer messages while preserving privacy.

In the openSCMS architecture, the ACA does not interact directly with clients. Instead, it is invoked exclusively by the Registration Authority (RA) as part of the authorization certificate provisioning flows. This design ensures that all client-facing validation, lifecycle checks, and policy enforcement are centralized in the RA, while the ACA remains narrowly focused on certificate generation.

As with other SCMS components, the ACA relies on the shared low-level cryptographic and protocol implementation provided by the C-based `oscms-codecs-bridge`. All certificate generation logic, including ASN.1 handling, IEEE 1609.2.1 encoding, and cryptographic operations, is performed via this shared library.

5.3.1 Role in Authorization Certificate Flows

Authorization certificate issuance is always initiated by the RA. When the RA receives an authorization certificate request from an EE, it validates the request, verifies the enrollment certificate, checks the EE lifecycle state, and extracts all parameters required for certificate issuance. The RA then constructs an internal task and forwards the relevant authorization issuance parameters to the ACA.

Before invoking the ACA, the RA interacts with the Linkage Authority (LA) to obtain the linkage values required for authorization certificate generation. These linkage values are used to enable misbehavior resolution and certificate revocation while preserving unlinkability during normal operation. Once the linkage values are available, the RA invokes the ACA to generate the authorization certificates.

The ACA generates the requested batch of authorization certificates and returns them to the RA. The RA persists the resulting certificates and makes them available to the client through the authorization download flow.

openSCMS supports the generation of both explicit and implicit authorization certificates, as defined by IEEE 1609.2.1.

Explicit certificates contain full public key material and signatures based on standard ECDSA over the P-256 curve. Implicit certificates rely on ECQV-style key derivation, significantly reducing certificate size by eliminating the need to transmit full public keys. Implicit certificates reduce the certificate size by approximately 64 bytes compared to their explicit counterparts.

The choice between explicit and implicit certificates is driven by policy and configuration parameters evaluated by the RA and passed to the ACA as part of the issuance request.

Authorization certificate issuance in openSCMS supports multiple butterfly key expansion mechanisms, which allow efficient generation of large batches of pseudonym certificates from compact input material.

The following butterfly mechanisms are supported:

- Original Butterfly Keys (OBK)
- Unified Butterfly Keys (UBK)
- Compact Unified Butterfly Keys (CUBK)

In addition to the butterfly expansion mechanisms, openSCMS supports both unencrypted and encrypted non-butterfly variantes. For encrypted inputs, the encryption key used to protect the butterfly material is the public encryption key of the RA. This ensures that only the RA is capable of decrypting sensitive key material received from the EE before it is forwarded for certificate generation.

5.3.2 Authorization Certificate Issuance Flow

During authorization certificate issuance, the ACA receives a request from the RA containing:

- the selected certificate type (explicit or implicit),
- the butterfly key expansion parameters,
- linkage values obtained from the LA,
- validity and permission constraints derived from policy.

Using this input, the ACA invokes the low-level C cryptographic engine to generate a batch of IEEE 1609.2.1 authorization certificates. All cryptographic operations, including key expansion, signature generation, and ASN.1 encoding, are performed within the C library to ensure consistency and correctness across components.

Once certificate generation completes, the ACA returns the resulting certificate batch to the RA. The ACA itself does not persist certificates or maintain client state.

5.3.3 Security and Architectural Considerations

The ACA is intentionally designed as a stateless and narrowly scoped service. It does not maintain EE lifecycle information, does not perform policy evaluation, and does not expose any externally accessible APIs. All trust decisions are delegated to the RA, which acts as the system's control plane.

By centralizing cryptographic implementation in a shared low-level library and limiting the ACA's responsibilities to certificate generation, openSCMS reduces implementation complexity, minimizes the attack surface, and facilitates auditing and reuse of the cryptographic core across all SCMS components.

6 oscms-codecs-bridge: Cryptographic Core and ASN.1 Codecs

The `oscms-codecs-bridge` library is the low-level cryptographic and protocol core of openSCMS. It is implemented in C and provides all security and critical functionality, including cryptographic operations, ASN.1 encoding and decoding, and construction and verification of IEEE 1609.2.1 SPDUs.

This library is designed as a standalone component, independent of the Rust backend and SCMS service logic. Integration with the Rust-based services is performed transparently through Rust bindings generated using `bindgen`. From the perspective of a Rust developer, the library appears as a native Rust module, while all cryptographic and protocol complexity remains encapsulated in C.

6.1 Build and Installation

The `oscms-codecs-bridge` library is built using CMake and follows a standard out-of-source build process.

Build Steps

```
cd oscms-codecs-bridge
mkdir build
cd build
cmake ..
make
make install
```

To install into a custom directory:

```
make DESTDIR=install install
```

Running Unit Tests The library includes an extensive suite of unit tests covering cryptographic primitives, ASN.1 encoding/decoding, and protocol message handling.

```
make test
```

6.2 High-Level Architecture

The architecture of `oscms-codecs-bridge` is layered to clearly separate codec specific logic from protocol processing and cryptographic operations.

At a high level, the library consists of:

- A codec abstraction layer (API)
- A concrete ASN.1 codec implementation (asn1c-based)
- A protocol bridge layer implementing IEEE 1609.2.1 semantics and Cryptographic utilities and primitives

This separation allows the library to remain codec-agnostic at higher layers, making it possible to support alternative ASN.1 transpilers in the future without modifying protocol or cryptographic logic.

6.3 Repository Structure

```
oscms-codecs-bridge/
├── codecs/                    # Codec implementations and abstractions
│   ├── asn1c/                  # asn1c-based IEEE 1609.2.1 codec
│   │   ├── src/                # Generated and handwritten codec glue
│   │   └── submodules/api/     # Abstract codec API
├── include/public/             # Public bridge headers
├── src/                        # Protocol and cryptographic bridge logic
├── scms_manager/               # Utility for certificate/key generation
├── tests/                      # Unit and integration tests
└── CMakeLists.txt
```

6.4 Codec Abstraction Layer

The codec abstraction layer defines a stable API that isolates the rest of the library from codec-specific data structures generated by ASN.1 transpilers. This API is located under:

```
codecs/submodules/api/
```

Concrete codec implementations (currently based on asn1c) implement this API and handle conversion between internal ASN.1 representations and abstract data structures exposed to the rest of the library.

For example, the ASN.1 representation of an elliptic curve point is defined by the transpiler as a deeply nested union-based structure. The codec API exposes an abstract and codec-independent representation:

ASN.1 Internal Representation (simplified)

```
typedef struct EccP256CurvePoint {
    ...
    union {
        OCTET_STRING_t x_only;
        struct {
            OCTET_STRING_t x;
            OCTET_STRING_t y;
        } uncompressedP256;
    } choice;
} EccP256CurvePoint_t;
```

Abstract API Representation

```
typedef struct OscmsEccCurvePoint {
    OscmsEccPointCurveType curve_type;
    OscmsEccPointType point_type;
    OscmsOctetBuffer x;
    OscmsOctetBuffer y;
} OscmsEccCurvePoint;
```

Conversion functions, such as `oscms_ecc_curve_point_from_internal`, translate between codec- specific and abstract representations. This pattern is consistently applied across all ASN.1-derived types.

6.5 Protocol Bridge Layer

The bridge layer implements the semantics of IEEE 1609.2.1 protocol messages and server-side flows. It provides functionality for:

- constructing and parsing SPDUs,
- signing and verifying messages,
- encrypting and decrypting payloads,
- validating certificate chains and trust anchors.

This layer is completely codec-agnostic and operates exclusively on abstract types provided by the API layer.

6.6 Memory Management Utilities

To ensure consistent and safe memory handling across the library, the API defines shared abstractions for buffers and sequences.

Octet Buffers The `OscmsOctetBuffer` type encapsulates a pointer-length pair and provides a unified interface for allocation, duplication, copying, and deallocation. This avoids error-prone handling of raw pointers and lengths.

```
OscmsOctetBuffer *oscms_octet_buffer_new_from_buffer(
    const uint8_t *buffer, size_t length);

void oscms_free_octet_buffer(OscmsOctetBuffer *buffer);
```

Sequences The `OscmsSequence` type provides a dynamic array abstraction similar to ASN.1 `SEQUENCE OF`. It is used to represent ordered collections of protocol elements while centralizing allocation and cleanup.

```
typedef struct OscmsSequence {
    size_t count;
    void **data;
} OscmsSequence;
```

6.7 Cryptographic Primitives

All cryptographic operations are implemented within the bridge layer using OpenSSL primitives. These include:

- ECDSA signing and verification,
- ECIES encryption and decryption,

- SHA-256 and SHA-384 hashing,
- implicit certificate key reconstruction,
- elliptic curve point compression and expansion.

For example, message signing is exposed via a simple API:

```
int ec_sign_message(
    const OscmsOctetBuffer *message,
    EVP_PKEY *private_key,
    OscmsHashAlgorithm hash_algorithm,
    uint8_t *signature);
```

These primitives are used uniformly by all SCMS components, ensuring consistent cryptographic behavior across the system.

6.8 Logging and Diagnostics

The library provides a lightweight logging abstraction that allows callers to integrate with their preferred logging system or fall back to standar output or syslog.

```
void oscms_log_set_logger(
    oscms_log_function_ptr func,
    int log_level);

void oscms_log(int level, const char *fmt, ...);
```

This allows the Rust backend to redirect all C-level logs into its own structured logging pipeline.

6.9 Integration with Rust

Integration with the Rust backend is achieved through the `oscms_bridge` Rust crate, which compiles the C library and generates bindings at build time using `bindgen`. This makes the C API available as safe Rust interfaces, while preserving the performance and security properties of the native implementation.

From the perspective of Rust service developers, all cryptographic and ASN.1 processing is accessed via normal Rust function calls, without requiring knowledge of the underlying C implementation.

6.10 Coding Standards and Style

Development in the `oscms-codecs-bridge` repository follows a strict and well-defined set of coding and style conventions. These conventions are largely derived from the Google C++ Style Guide (`https://google.github.io/styleguide/cppguide.html`), adapted to account for the fact that the implementation language is primarily C rather than C++.

Code formatting is controlled by the repository's `.clang-format` file and enforced using version 18 of the `clang-format` tool.

6.10.1 C Language Standard and Tooling

- The code is written according to the C11 standard.
- POSIX-compliant APIs are preferred when available (for example, using `stat` instead of `access`).
- GCC version 11 or newer is required.
- CMake version 3.31.5 is used, matching the version supported by Ubuntu 22.04.

6.10.2 Header Files

The following conventions apply to all header files:

- All headers must be self-contained.
- Include guards must be implemented using `#define`; trailing underscores are not permitted.
- The "include what you use" principle must be followed.
- Forward declarations are used when necessary, primarily:
  - to declare opaque types, or
  - to allow a `struct` to contain a pointer-to-self member.
- Inline functions should be used sparingly and must not expose internal implementation details that would otherwise remain opaque.
- Inline functions are always preferred over parameterized `#define` macros.

Include directives must follow a strict ordering, with groups separated by a blank line:

- system headers (angle brackets),
- intra-project headers from other components,
- headers from the same component.

6.10.3 Scoping

Namespaces Although namespaces are a C++ feature and therefore not applicable in C, related functions must still be named consistently. A shared prefix should be used for all public identifiers to avoid symbol collisions.

Linkage

- All internal functions within a module must be declared `static`.
- Functions callable by other modules within the same library use normal external linkage.
- Functions that form part of a library's public API must be annotated with the `SO_EXPORT` qualifier in both header and implementation files.

Example:

```
// x.h
#include "so_export.h"

void bar();               // internal to the library
SO_EXPORT void bang();    // public API

// x.c
static void foo() {}      // internal to this file
void bar() {}
SO_EXPORT void bang() {}
```

6.10.4 Local and Static Variables

Local variables must:

- use the narrowest possible scope,
- be initialized at declaration,
- avoid cryptic or ambiguous names.

Static storage should be avoided in multithreaded environments, as it introduces implicit shared state. The only acceptable exception is write-once, read-many data.

6.10.5 Structs

C does not provide classes, but similar encapsulation can be achieved using `struct` definitions.

Declarations The following declaration style is strongly preferred:

```
typedef struct {
    // members
} StructTypeName;
```

This allows natural usage without repeatedly referring to `struct`. The commonly used `_t` suffix is deliberately avoided, as it is reserved by POSIX.

Opaque vs. Exposed Structs Opaque types are preferred when the API operates primarily on objects of a given type. In this pattern, the header contains only a forward declaration, and the full definition is kept private within the implementation file:

```
// x.h
typedef struct implementation implementation;

implementation *implementation_new();
void do_something_with(implementation *thing);
void implementation_free(implementation *thing);
```

```
// x.c
struct implementation {
    // internal fields
};
```

This approach allows internal representations to evolve without breaking API compatibility, at the cost of requiring explicit memory management by clients.

When exposing a `struct` directly, functions must never accept or return structures by value. All interactions must use pointers.

6.10.6 Naming Conventions

The following naming conventions are enforced consistently:

- Source file names: `snake_case`
- `typedef`, `struct`, and `enum` types: `PascalCase`
- `#define` constants and enum values: `UPPER_SNAKE_CASE`
- Functions: `snake_case`
- Variables: `snake_case`

6.11 Extensibility Considerations

Although the current implementation supports only an `asn1c`-based codec and server-side SPDU coverage, the architecture is intentionally designed to be extensible.

Additional ASN.1 transpilers, client-side decoding support, or new cryptographic mechanisms can be introduced by implementing the codec API and extending the bridge layer without disrupting existing functionality.

7 Testing and Validation

openSCMS adopts a multi-layered testing strategy to ensure correctness, interoperability, and security across its cryptographic core, service implementations, and end-to-end protocol flows. Validation is performed through a combination of unit tests at the C and Rust levels, as well as end-to-end (E2E) integration tests using a certified V2X client.

This layered approach ensures that:

- low-level cryptographic primitives and ASN.1 codecs behave correctly,
- service-level logic is validated in isolation,
- complete IEEE 1609.2.1 protocol flows operate as expected in realistic deployment scenarios.

7.1 Unit Tests: oscms-codecs-bridge (C)

The `oscms-codecs-bridge` library includes an extensive suite of unit tests implemented using CMake/CTest. These tests focus on correctness, robustness, and edge-case handling of cryptographic primitives, ASN.1 encoding and decoding, and protocol SPDU construction.

A total of **121 unit tests** are executed as part of the library test suite, covering both success and failure scenarios.

Test Execution

```
cd oscms-codecs-bridge/build
make test
```

Coverage Overview

Category	Description	Tests
Memory Utilities	Octet buffers, sequences, tracked allocation	20+
ASN.1 Codecs	Encoding/decoding of IEEE 1609.2.1 SPDUs	30+
Cryptographic Primitives	ECDSA, ECIES, hashing, implicit cert derivation	25+
Certificate Handling	Certificate chains, CRLs, CTL, CMIS	20+
Protocol Handlers	RA, ECA, ACA request/response processing	25+

Representative Validated Behaviors The following behaviors are explicitly validated by unit tests:

- round-trip ASN.1 encoding and decoding of SPDUs,
- validation of malformed or incomplete protocol messages,
- cryptographic signature generation and verification,
- encryption and decryption using ECIES,
- generation of authorization certificates using OBK, UBK, and CUBK butterfly mechanisms,
- correct handling of encrypted and non-encrypted butterfly inputs.

All tests complete successfully with zero failures, confirming deterministic and standards-compliant behavior of the cryptographic core.

7.2 Unit Tests: Rust Services

In addition to C-level testing, openSCMS includes unit tests for Rust service components. These tests validate higher-level logic, orchestration, and data processing that sits above the cryptographic bridge.

Rust Test Execution

```
cargo test
```

Coverage Overview

Module	Focus Area	Tests
Core Logic	Linkage seeds, PLVs, butterfly expansion	10+
Bridge Integration	SPDU construction and parsing	14+
RA Endpoints	Authorization, validation, worker flows	15+
Utility Modules	Hashing, ID generation, helpers	5+

Validated Properties Rust unit tests ensure that:

- cryptographic outputs from the C bridge are correctly interpreted,
- linkage values and butterfly expansions are consistent and reproducible,
- request identifiers, hashes, and filenames follow protocol rules,
- RA worker logic correctly orchestrates calls to ACA and LA components.

All Rust unit tests complete successfully with no failures.

7.3 End-to-End (E2E) Validation with V2X Client

openSCMS was validated using a certified V2X client implementing IEEE 1609.2.1. These end-to-end tests exercise complete protocol flows across SCMS services, cryptographic trust anchors, persistent storage, and messaging infrastructure.

The table below summarizes the validated E2E scenarios and outcomes.

Scenario	Result	Notes / Output
Register canonical public key (uncompressed)	SUCCESS	Returns canonical ID and device ID
Register already registered canonical key	ERROR	EE registration already exists
Register canonical key with invalid length or format	ERROR	Invalid public key length or format
Register canonical key not starting with `0x04`	ERROR	Public key is not in uncompressed format
Request enrollment certificate using registered canonical key (implicit)	SUCCESS	Enrollment certificate issued
Request enrollment certificate with unregistered canonical ID	ERROR	Device is not registered
Request enrollment certificate without canonical ID	ERROR	Canonical ID is required
Request authorization certificate (non-butterfly, plain; explicit or implicit)	SUCCESS	Single authorization certificate
Request authorization certificate (non-butterfly, encrypted; explicit or implicit)	SUCCESS	Single authorization certificate
Request authorization certificate (OBK butterfly; explicit or implicit)	SUCCESS	Batch of authorization certificates
Request authorization certificate (UBK butterfly; explicit or implicit)	SUCCESS	Batch of authorization certificates
Request authorization certificate (CUBK butterfly; explicit or implicit)	SUCCESS	Batch of authorization certificates
Download Root CA, ICA, ECA, ACA certificates	SUCCESS	Certificates retrieved successfully
Download RA certificate	SUCCESS	RA certificate retrieved successfully
Download Certificate Trust List (CTL)	SUCCESS	CTL file retrieved
Download Certificate Chain File (CCF)	SUCCESS	CCF file retrieved
Download Certificate Revocation List (CRL)	SUCCESS	CRL file retrieved
Download Certificate Management Info Status (CMIS)	SUCCESS	CMIS file retrieved

7.4 Validation Summary

The combination of extensive unit testing and comprehensive end-to-end validation demonstrates that openSCMS:

- correctly implements IEEE 1609.2.1 cryptographic and protocol semantics,
- handles error conditions deterministically and securely,
- scales from low-level primitives to full-system workflows,
- is suitable for use in development, testing, and evaluation environments.

This testing strategy also provides a strong foundation for future extensions, optimizations, and certification efforts.

8 Future Directions and Contributions

openSCMS provides a solid, standards-compliant foundation for secure V2X infrastructure, but several areas remain open for extension and research.

Potential future directions include:

- Misbehavior detection and reporting components
- Advanced CRL generation and distribution strategies
- Performance benchmarking and optimization tooling
- Additional client-side reference implementations

Engineers are encouraged to contribute by extending existing services, improving documentation, adding test coverage, or experimenting with alternative SCMS design choices.

By remaining open, modular, and standards-aligned, openSCMS aims to serve as both a production-ready reference implementation and a research platform for the future of secure V2X communication.

A Deployment Parameters

```
# Deployment environment
DEPLOY_ENVIRONMENT=minikube

# Logging
RUST_LOG=debug

# SCMS V2/V3 error handling
SCMS_V2_ERROR=fine

# Service ports
ACA_PORT=5000
CAM_PORT=5000
ECA_PORT=5000
RA_PORT=5000
LA_PORT=5000

# LA configuration
LA_COUNT=2

# Certificate generation parameters
CERTIFICATES_PER_BATCH=10
NUMBER_CERT_BATCHES=1
MIN_NUMBER_CERTS_RA=1

# Time and period configuration
PERIOD_LENGTH_DAYS=1
TIME_I_PERIOD_EPOCH=0
TIME_I_PERIOD_INIT=0
```

```
# CTL configuration
CTL_SERIES_ID=FFFFFFFFFFFFFF00

# Certificate and key paths
ACA_CERTS_FOLDER=/certs/aca
ACA_CERTIFICATE_FILE=acaCertificate.coer
ACA_PRIVATE_KEY_FILE=acaPrivateKey.dat
ACA_PUBLIC_KEY_FILE=acaPublicKey.dat

RA_CERTS_FOLDER=/certs/ra
RA_CERTIFICATE_FILE=raCertificate.coer
RA_PRIVATE_KEY_FILE=raPrivateKey.dat
RA_PUBLIC_KEY_FILE=raPublicKey.dat
RA_ENC_PUBLIC_KEY_FILE=raEncPublicKey.dat
RA_ENC_PRIVATE_KEY_FILE=raEncPrivateKey.dat

ECA_CERTS_FOLDER=/certs/eca
ECA_CERTIFICATE_FILE=ecaCertificate.coer
ECA_PRIVATE_KEY_FILE=ecaPrivateKey.dat
ECA_PUBLIC_KEY_FILE=ecaPublicKey.dat

ICA_CERTS_FOLDER=/certs/ica
ICA_CERTIFICATE_FILE=icaCertificate.coer

ROOTCA_CERTS_FOLDER=/certs/rootca
ROOTCA_CERTIFICATE_FILE=rootcaCertificate.coer

CTL_FOLDER=/certs/ctl
CTL_FILE=ctl.coer
CTL_CCF_FILE=ccf.coer

# Protocol files
ROOT_PROTOCOL_FILES_PATH=/protocol_files
RA_PROTOCOL_FILES_FOLDER=protocol_files
RA_RESPONSE_TO_EE_FOLDER=ra_to_ee_files
RA_RESPONSE_TO_EE_ZIP_FOLDER=ra_to_ee_files_zip

# System parameters (IEEE 1609.2.1)
PARAM_WEBAPI_NAME=scmsV3
PARAM_WEBAPI_EE_AUTH=None
PARAM_SCMSV3_EE_AUTH=None
PARAM_SCMSV3_ERROR=fine
PARAM_SCMSV3_OPTIONS=

PARAM_ECA_MAX_AGE=1000
PARAM_ECA_MAX_REQS=100
PARAM_ECA_MAX_WAIT=100
PARAM_ECA_MIN_WAIT=0
```

```
PARAM_RA_ACPC_SUPPORT=true
PARAM_RA_BUTTERFLY_TYPE="compact unified"
PARAM_RA_MAX_AGE=0
PARAM_RA_MAX_GEN_DELAY=4000
PARAM_RA_MAX_REQS=100
PARAM_RA_MAX_RELOAD_TIME=0
PARAM_RA_MIN_WAIT=0

PARAM_DOWNLOAD_MAX_AGE=1000
PARAM_DOWNLOAD_MAX_REQS=100
PARAM_DOWNLOAD_MIN_WAIT=0

PARAM_SUCCESSOR_NEXT_DL_TIME=5
PARAM_CERT_NEXT_DL_TIME=5

RA_ID=1
RA_PATH=RA
```

B Device-Specific Policy File

This appendix presents a complete example of the device-specific policy file returned by the Registration Authority when a request includes an EE identifier such as `canonicalId` or `deviceId`.

```
{
  "ee-scmsV3-eeAuth": {
    "enrollment-certificate": [
      "Canonical"
    ],
    "authorization-certificate-request": [
      "Enrollment"
    ],
    "authorization-certificate-download": [
      "Enrollment"
    ],
    "authorization-certificate-download-filename": [
      "Enrollment"
    ],
    "ccf-ctl": [
      "NoAuth"
    ],
    "composite-ccf-ctl": [
      "NoAuth"
    ],
    "ca-certificate": [
      "NoAuth"
    ],
    "crl": [
      "NoAuth"
    ],
```

```
    "ctl": [
      "NoAuth"
    ],
    "ra-certificate": [
      "NoAuth"
    ],
    "certificate-management-info-status": [
      "NoAuth"
    ],
    "successor-enrollment-certificate-request": [
      "Enrollment"
    ],
    "successor-enrollment-certificate-download": [
      "Enrollment"
    ],
    "misbehavior-report": [
      "Authorization"
    ],
    "ma-certificate": [
      "NoAuth"
    ]
  },
  "ee-ra-butterflyType": "Original",
  "ee-download-url": "",
  "ee-supportPsidSsp": [
    {
      "all": []
    }
  ],
  "ee-supportRegion": [
    {
      "region": [
        {
          "country": [],
          "countryAndRegion": [],
          "countryAndRegionAndSubRegion": [],
          "circle": []
        }
      ]
    }
  ],
  "ee-countCert": 1,
  "ee-heartbeatInterval": 0
}
```


C Global SCMS Policy File

This appendix presents a complete example of the global SCMS policy file returned by the Registration Authority when no EE identifier is provided in the request.

```
{
  "quorumCount": 3,
  "ctlSeriesId": "FFFFFFFFFFFFFF00",
  "cryptoAlgorithm": [
    "nistP256",
    "nistP384",
    "brainpoolP256",
    "brainpoolP384",
    "sha256",
    "sha384",
    "aes128Ccm"
  ],
  "certElector": {
    "name": "elector",
    "cryptoAlgorithm": [
      "nistP384",
      "sha384"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certRootCa": {
    "name": "root-ca",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certIca": {
    "name": "ica",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
```

```
  },
  "certEca": {
    "name": "eca",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certAca": {
    "name": "aca",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certPca": {
    "name": "pca",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certCrlSigner": {
    "name": "crl-signer",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
```

```
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certRa": {
    "name": "ra",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certLa": {
    "name": "la",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certLa1": {
    "name": "la1",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certLa2": {
    "name": "la2",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
```

```
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certMa": {
    "name": "ma",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certPg": {
    "name": "pg",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certDc": {
    "name": "dc",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Explicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certEnrollment": {
    "name": "",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
```

```
    "certificateType": "Implicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certPseudonym": {
    "name": "",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Implicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "certIdentification": {
    "name": "",
    "cryptoAlgorithm": [
      "nistP256",
      "sha256"
    ],
    "certificateType": "Implicit",
    "start": 0,
    "expiration": 0,
    "inUse": 0,
    "requestForRenewal": 0,
    "concurrentlyValidCertificates": 1
  },
  "supportRegion": [
    {
      "country": [
        "840",
        "124",
        "484",
        "410",
        "724",
        "156"
      ],
      "countryAndRegion": [],
      "countryAndRegionAndSubRegion": [],
      "circle": []
    }
  ],
  "supportPsidSsp": [
    {
```

```
      "all": []
    }
  ],
  "urlDomain": "192.168.49.2",
  "ecaHostname": "192.168.49.2:30085",
  "raHostname": "192.168.49.2:30080",
  "dcHostname": "",
  "dcmHostname": "",
  "ecaUrl": "http://192.168.49.2:30085/v3/",
  "raUrl": "http://192.168.49.2:30080/v3/",
  "dcUrl": "http://192.168.49.2:30080/v3/",
  "dcmUrl": "http://192.168.49.2:30080/",
  "time-iPeriod": 8022,
  "time-iPeriodEpoch": 0,
  "time-iPeriodInit": 0,
  "time-iPeriodLength": 86400,
  "time-overlap": 0,
  "time-baseCertNumber": 10,
  "time-overlapCertNumber": 0,
  "session-scmsAuth": "none",
  "session-eeAuth": "NoAuth",
  "webApi-name": "scmsV3",
  "webApi-eeAuth": "NoAuth",
  "scmsV3-eeAuth": {
    "enrollment-certificate": [
      "Canonical"
    ],
    "authorization-certificate-request": [
      "Enrollment"
    ],
    "authorization-certificate-download": [
      "Enrollment"
    ],
    "authorization-certificate-download-filename": [
      "Enrollment"
    ],
    "ccf-ctl": [
      "NoAuth"
    ],
    "composite-ccf-ctl": [
      "NoAuth"
    ],
    "ca-certificate": [
      "NoAuth"
    ],
    "crl": [
      "NoAuth"
    ],
    "ctl": [
      "NoAuth"
```

```
    ],
    "ra-certificate": [
      "NoAuth"
    ],
    "certificate-management-info-status": [
      "NoAuth"
    ],
    "successor-enrollment-certificate-request": [
      "Enrollment"
    ],
    "successor-enrollment-certificate-download": [
      "Enrollment"
    ],
    "misbehavior-report": [
      "Authorization"
    ],
    "ma-certificate": [
      "NoAuth"
    ]
  },
  "scmsV3-error": "Fine",
  "scmsV3-options": [
    ""
  ],
  "eca-maxAge": 1000,
  "eca-maxReqs": 100,
  "eca-maxWait": 100,
  "eca-minWait": 0,
  "ra-acpcSupport": [],
  "ra-butterflyType": {
    "original": [
      "32",
      "38"
    ],
    "unified": [],
    "compactUnified": []
  },
  "ra-maxAge": 0,
  "ra-maxGenDelay": 4000,
  "ra-maxReqs": 100,
  "ra-maxPreloadTime": 0,
  "ra-minWait": 0,
  "download-maxAge": 1000,
  "download-maxReqs": 288,
  "download-minWait": 0,
  "time-identificationCertLength": 0,
  "time-identificationCertOverlap": 0,
  "time-leapSeconds": 0,
  "webApi-connectionRetries": 3,
  "webApi-betweenRetries": 1000,
```

```
"webApi-betweenRetries-infoStatus": 1000,
"webApi-toleranceTime": 1000,
"nextUpdate-timeDefault": {
  "fullHashCrl": 0,
  "deltaHashCrl": 0,
  "fullLinkedCrl": 0,
  "deltaLinkedCrl": 0,
  "fullRootCtl": 0,
  "deltaRootCtl": 0,
  "fullDomainCtl": 0,
  "deltaDomainCtl": 0
},
"supportRegion-obuDefault": [
  {
    "region": [
      {
        "country": [],
        "countryAndRegion": [],
        "countryAndRegionAndSubRegion": [],
        "circle": []
      }
    ]
  }
],
"supportRegion-rsuDefault": [
  {
    "region": [
      {
        "country": [],
        "countryAndRegion": [],
        "countryAndRegionAndSubRegion": [],
        "circle": []
      }
    ]
  }
],
"supportPsidSsp-obuDefault": [
  {
    "all": []
  }
],
"supportPsidSsp-rsuDefault": [
  {
    "all": []
  }
],
"scmsV3-eeAuth-default": {
  "enrollment-certificate": [
    "Canonical"
  ],
```

```
    "authorization-certificate-request": [
      "Enrollment"
    ],
    "authorization-certificate-download": [
      "Enrollment"
    ],
    "authorization-certificate-download-filename": [
      "Enrollment"
    ],
    "ccf-ctl": [
      "NoAuth"
    ],
    "composite-ccf-ctl": [
      "NoAuth"
    ],
    "ca-certificate": [
      "NoAuth"
    ],
    "crl": [
      "NoAuth"
    ],
    "ctl": [
      "NoAuth"
    ],
    "ra-certificate": [
      "NoAuth"
    ],
    "certificate-management-info-status": [
      "NoAuth"
    ],
    "successor-enrollment-certificate-request": [
      "Enrollment"
    ],
    "successor-enrollment-certificate-download": [
      "Enrollment"
    ],
    "misbehavior-report": [
      "Authorization"
    ],
    "ma-certificate": [
      "NoAuth"
    ]
  },
  "ra-butterflyType-default": "Original",
  "ra-encryptionKey-default": "encryptionKey",
  "download-url-default": "",
  "count-certPseudonym-default": 1,
  "count-certIdentification-default": 1,
  "ee-heartbeatInterval": 0
}
```